

Koşullu Tekrarlar ve Kontrol Akışı

Görsel Programlama - Hafta 12



Haftaya Giriş: Koşullu Tekrarlar ve Kontrol Akışı

Bu hafta, C# programlamada koşula dayalı tekrar yapılarını detaylıca inceleyeceğiz.

****while**** ve ****do-while**** döngülerinin temel ayrımını ve kullanım senaryolarını analiz edeceğiz.

Ayrıca, Windows Forms uygulamalarının temel bileşenlerinden biri olan ****ComboBox**** kontrolünü tanıyacağız.

Hedefimiz, program akışını koşullu olarak kontrol edebilme ve kullanıcı arayüzü etkileşimini sağlama yetkinliğini kazanmaktır.



C# Döngülerine Kısa Bir Bakış

Döngüler, aynı kod bloğunu belirli koşullar altında veya belirli sayıda çalıştırmak için kullanılır.

Önceki derslerde ****for**** döngüsünü görmüştük (sayaç tabanlı döngü). ****while**** ve ****do-while**** döngüleri ise, kaç kez çalışacakları kesin olarak bilinmeyen veya bir boolean koşuluyla bağlı olan durumlarda tercih edilir. Bu döngüler, programın esnekliğini ve verimliliğini artırır.



while Döngüsü: Tanım ve Çalışma Prensipleri

while döngüsü, 'koşul doğru olduğu sürece' belirli bir kod bloğunu tekrarlar. Bu döngü, bir ****ön test döngüsü**** (pre-test loop) olarak bilinir, yani koşul döngü bloğuna girmeden önce kontrol edilir.

****Anahtar Kural:**** Koşul en başta yanlışsa, döngü bloğu ****hiç çalışmayabilir****.

while Döngüsü Söz Dizimi (Syntax) ve Akış Şeması

Söz Dizimi:

```
'while (koşul)'  
{  
  '// Koşul doğru olduğu sürece çalışacak kod'  
}
```

Koşul, daima 'true' veya 'false' döndüren bir boolean ifade olmalıdır.

Döngünün sonsuz döngüye girmemesi için, döngü içindeki kod bloğunda koşulu etkileyecek bir değişikliğin yapılması hayati önem taşır.

while Döngüsü Kullanım Senaryoları

while döngüsü, tekrar sayısının program çalıştığında kesinleşmediği durumlarda kullanılır.

****Örnekler:****

1. Dosya Okuma: Dosyanın sonuna (End-of-File) ulaşılan kadar satırları okuma.
2. Ağ Bağlantısı: Bağlantı kurulana kadar yeniden deneme.
3. Kullanıcı Girişi: Belirli bir komut ('çıkış') girilene kadar programın çalışması.

while Uygulaması: Basit Sayıcı

```
int i = 1;
while (i <= 5)
{
    Console.WriteLine(i);
    i++; // Koşulun yanlış olması için kritik adım
}
```

****Önemli Not:**** Eğer 'i++;' satırı unutulursa, koşul sürekli doğru kalır ve ****sonsuz döngü**** (infinite loop) oluşur.



Sonsuz Döngüler (Infinite Loops) ve Önlenmesi

Bir döngünün koşulu asla yanlış olmuyorsa, bu bir sonsuz döngüdür ve programın kilitlenmesine neden olabilir (Form uygulamalarında özellikle tehlikelidir).

****Nedenleri:****

1. Koşulun yanlış olmasını sağlayacak değişkenin güncellenmemesi.
2. Koşulun yanlış hesaplanması (örneğin, tam sayı aritmetiği hataları).

****Çözüm:**** Döngü içerisinde 'break' anahtar kelimesini kullanarak döngüyü koşulsuz olarak sonlandırmak ('while(true)' yapılarında yaygındır).

do-while Döngüsü: Tanım ve Ayırt Edici Özellik

do-while döngüsü de bir koşul doğru olduğu sürece tekrar eder. Ancak, bu döngü bir ****son test döngüsü**** (post-test loop) olarak çalışır. ****Anahtar Özellik:**** Koşul döngü bloğu çalıştırıldıktan sonra kontrol edildiği için, do-while döngüsü ****en az bir kez**** çalışmayı garanti eder. Bu, kullanıcı etkileşimli sistemler için hayati bir özelliktir.



do-while Döngüsü Söz Dizimi (Syntax)

Söz Dizimi:

```
'do'
```

```
{
```

```
'// Kod bloğu, koşul ne olursa olsun en az bir kez çalışır'
```

```
}
```

```
'while (koşul);'
```

****Uyarı:**** while kelimesinden sonra noktalı virgül (;) kullanılması zorunludur.

Bu, while döngüsünden temel bir sintaks farkıdır.

do-while Uygulaması: En Az Bir Kez Çalışma Garantisi

```
int i = 10; // Başlangıç koşulunu sağlayan bir değer
```

```
do
```

```
{
```

```
    Console.WriteLine('Çalıştım: ' + i);
```

```
    i++;
```

```
}
```

```
while (i < 5);
```

****Sonuç:**** Koşul ($i < 5$) baştan yanlıştır ($10 < 5$), ancak döngü bir kez çalışır. Bu, 'while' döngüsünde asla gerçekleşmeyecek bir durumdur.

do-while İçin Temel Kullanım Alanı: Girdi Doğrulama

do-while, özellikle kullanıcıdan geçerli bir girdi (örneğin pozitif bir sayı, doğru bir şifre) alana kadar sormaya devam etme mantığında kullanılır.

1. ****do:**** Kullanıcıdan girdiyi al ve işlemi yap.
2. ****while (koşul):**** Girdi geçerli olmadığı sürece (koşul doğru olduğu sürece) tekrar et.

Bu yapı, kullanıcı etkileşimli programlarda temiz ve döngüsel bir doğrulama sağlar.



while vs. do-while: Karşılaştırmalı Analiz

while (Ön Test): Koşul doğru olduğu sürece döner, hiç çalışmayabilir.

Kullanım Alanı: Koleksiyonları işleme, dosya/stream sonunu kontrol etme, giriş yapılması zorunlu olmayan durumlar.

do-while (Son Test): Koşul doğru olduğu sürece döner, en az bir kez çalışır.

Kullanım Alanı: Kullanıcıdan zorunlu girdi alma, menü seçimi, ilk denemesi garantilenen işlemler (örneğin deneme hakkı senaryoları).

Döngü Kontrol Anahtar Kelimeleri: break

'break' anahtar kelimesi, döngünün (for, while, do-while veya switch) o anki iterasyonunu değil, döngünün kendisini tamamen sonlandırır. Özellikle 'while(true)' yapılarında, belirli bir iç koşul sağlandığında döngüyü güvenli bir şekilde terk etmek için kullanılır. Bu, döngü sonlandırma koşulunun döngü gövdesi içinde karmaşık hesaplamalara bağlı olduğu durumlarda kullanışlıdır.

Döngü Kontrol Anahtar Kelimeleri: continue

'continue' anahtar kelimesi, döngünün geri kalan kod bloğunu atlar ve hemen bir sonraki iterasyonun başlangıcına, yani koşul kontrolüne döner.

'for' döngülerinde sayaç artırımı (increment) 'continue'dan sonra yapılır.

'while' ve 'do-while' döngülerinde ise, 'continue' komutundan önce koşul değişkeninin güncellendiğinden emin olunmalıdır; aksi takdirde sonsuz döngü riski vardır.

Gerçek Dünya Senaryosu: Güvenlik Doğrulaması

Şifre doğru girilene kadar sorma senaryosu: Teorik olarak 'do-while' mantığına uyar ('en az bir kez sor').

****Form Uygulamalarında Zorluk:**** C# WinForms'ta, 'while' döngüsü bir 'Button click' olayı içinde çalıştırılırsa, döngü tamamlanana kadar form donar (UI kilitlenir).

Bu nedenle, form projelerinde genellikle döngü yerine 'hak sayacı' tutan değişkenler ve 'Button click' olayları kullanılır.



UI Kilitlenmesi (Freezing UI) Problemi

Windows Forms uygulamaları genellikle tek bir ana iş parçacığında (UI thread) çalışır.

Bu iş parçacığı, hem kullanıcı arayüzünü çizer hem de tüm olayları (buton tıklamaları, fare hareketleri) işler.

Uzun süren bir 'while' döngüsü bu iş parçacığını meşgul eder ve arayüzün yanıt vermemesine neden olur.

****Çözüm Yolu:**** Eşzamanlı programlama (Asenkron veya Multithreading) tekniklerini kullanarak döngüleri arka plana taşımaktır.

Proje 1 Analizi: Şifre Doğru Girilene Kadar Sor

do-while yerine WinForms'ta daha pratik olan hak sayacı (`int hak = 3`) mantığı.

1. Kullanıcı butona basar.

2. 'if (şifre yanlış ise)': 'hak' değişkeni bir azaltılır ('hak--').

3. 'if (hak == 0)': Kullanıcıya 'Hakkınız bitti' mesajı verilir ve giriş engellenir.

Bu yaklaşım, döngü kullanmadan do-while mantığının (tekrar deneme) eşdeğerini olay tabanlı programlamaya uyarlar.

İç İçe Döngüler (Nested Loops) Kavramı

İç içe döngüler, bir döngünün gövdesinde başka bir döngünün yer almasıdır.
Kullanım Alanı: Matrisler veya iki boyutlu diziler gibi hiyerarşik veya çok boyutlu veri yapılarını işlerken zorunludur.
Genellikle dış döngü satırları, iç döngü ise sütunları temsil eder.
while veya do-while döngüleri, for döngüleri ile iç içe geçebilir.



Döngü Performansı ve Verimlilik

Üniversite düzeyinde programlamada döngülerin performansı önemlidir. Çoğu temel döngü (while, do-while, for) $O(n)$ zaman karmaşıklığına sahiptir (n eleman sayısına bağlı).

Ancak, iç içe döngüler genellikle $O(n^2)$ veya daha yüksek karmaşıklıkta olabilir.

Büyük veri setleriyle çalışırken, döngü yapısının doğru seçilmesi ve gereksiz iterasyonlardan kaçınılması önemlidir.



Yeni Form Kontrolü: ComboBox (Açılır Liste)

ComboBox, kullanıcıların önceden belirlenmiş bir listeden bir öge seçmesine olanak tanıyan bir UI kontrolüdür.

****Avantajları:****

1. Alan Tasarrufu: Liste varsayılan olarak kapalıdır, form üzerinde az yer kaplar.
2. Standardizasyon: Kullanıcının sadece tanımlanan geçerli girdileri seçmesini sağlar.

Türkçe'de genellikle 'Açılır Liste' veya 'Kombo Kutu' olarak adlandırılır.

ComboBox Temelleri: Items Koleksiyonu

ComboBox kontrolü, seçeneklerini tutmak için bir 'Items' koleksiyonuna sahiptir.

Bu koleksiyon, nesneleri (genellikle dizeleri) depolayabilir.

****Öğe Ekleme:**** En basit yöntem, 'Items.Add()' metodunu kullanmaktır.

Örnek: 'comboBox1.Items.Add('Öğe Adı');'

Bu işlem, genellikle form yüklenirken (Form_Load) veya bir olayın tetiklenmesiyle yapılır.

ComboBox Özellikleri: Text

'Text' özelliği, ComboBox'ın o anda görünen metnini temsil eder. Eğer ComboBox stili kullanıcı girişine izin veriyorsa, kullanıcının yazdığı metin bu özellikten okunur. Eğer liste içinden bir öğe seçiliyse, seçilen öğenin metinsel değeri bu özellikten alınır. Bu özellik, genellikle kullanıcının ne seçtiğini veya ne yazdığını hızlıca öğrenmek için kullanılır.

ComboBox Özellikleri: SelectedIndex

'SelectedIndex' özelliği, listede seçili olan öğenin sıfır tabanlı dizinini (index) döndürür.

Liste boşsa veya hiçbir öğe seçili değilse, bu özellik '-1' değerini döndürür.

Programatik olarak bir öğe seçmek için de bu özellik kullanılabilir:

'comboBox1.SelectedIndex = 2;' (listenin üçüncü öğesini seçer).

ComboBox Özellikleri: SelectedItem

'SelectedItem' özelliği, listede seçili olan öğenin kendisini (bir nesne olarak) döndürür.

'Items.Add()' metoduyla eklenen nesnelerin tipini döndürür; eğer string eklediysek, 'SelectedItem' da bir 'object' olarak döner ve string'e dönüştürülmelidir (casting).

Bu özellik, özellikle liste nesneler (örneğin Şehir nesneleri) içerdiğinde, nesnenin tüm detaylarına erişim sağlar.



Proje 2: ComboBox'ı Şehirlerle Doldurma (Statik)

Kaynakta belirtilen statik doldurma örneği:

```
'comboBox1.Items.Add('İstanbul');'  
'comboBox1.Items.Add('Ankara');'  
'comboBox1.Items.Add('İzmir');'  
'comboBox1.Items.Add('Giresun');
```

Bu yöntem küçük ve sabit listeler için uygundur, ancak dinamik veri kaynakları için verimsizdir.

Bu kod akışı genellikle 'Form_Load' olayı içinde yer alır, böylece uygulama başladığında liste hazır olur.

ComboBox'ı Dinamik Doldurma Kavramı

Gerçek dünya uygulamalarında, seçenekler bir diziden ('string[]'), listeden ('List<string>') veya bir veritabanından gelir.

Bu durumda, her bir öğeyi tek tek 'Items.Add()' ile eklemek yerine, döngüler kullanırız.

Bu, kodu daha sürdürülebilir ve esnek hale getirir.

foreach Döngüsü ile ComboBox Doldurma

'foreach' döngüsü, bir koleksiyondaki tüm öğeleri tek tek dolaşmak için idealdir.

```
string[] sehirler = {'İstanbul', 'Ankara', 'İzmir'};
```

```
'foreach(string sehir in sehirler)'
```

```
'{'
```

```
'    comboBox1.Items.Add(sehir);'
```

```
'}'
```

Bu yöntem, 'for' döngüsüne göre daha temiz ve okunabilir bir kod sağlar, özellikle dizinin veya listenin boyutuyla ilgilenmek zorunda kalmayız.

for Döngüsü ile ComboBox Doldurma

'for' döngüsü, dizin bazlı (index-based) erişim gerektiren durumlarda veya listenin boyutunu bilmemiz gerektiğinde kullanılabilir.

```
string[] sehirler = {'İstanbul', 'Ankara', 'İzmir'};
```

```
'for (int i = 0; i < sehirler.Length; i++)'
```

```
'{'
```

```
'    comboBox1.Items.Add(sehirler[i]);'
```

```
'}'
```

Bu yöntem, elemanlara belirli bir sırayla erişim veya sadece listenin bir kısmını ekleme ihtiyacı olduğunda önemlidir.

while Döngüsü ile Dinamik Doldurma

while döngüsü, özellikle bir veritabanı okuyucusundan (DataReader) veya bir stream'den veri okurken ComboBox'ı doldurmak için kullanılabilir.

```
'while (dataReader.Read())'  
{  
'comboBox1.Items.Add(dataReader['SehirAdi'].ToString());'  
}'
```

Bu senaryoda, döngü, okuyucunun verisi kalmayana kadar (koşul yanlış olana kadar) çalışmaya devam eder.

ComboBox Olayları: SelectedIndexChanged

Kullanıcı listede bir seçim yaptığında, 'SelectedIndexChanged' olayı tetiklenir. Bu olay işleyicisi (event handler), seçimin hemen ardından programın bir eylem gerçekleştirmesi için kullanılır.

Örnek: Bir şehir seçildiğinde, o şehrin nüfusunu veya posta kodunu başka bir metin kutusunda göstermek.



while Döngüsünde Gelişmiş Kontrol: Zaman Aşımı (Timeout)

Ağ işlemleri veya kaynak bekleme durumlarında, sonsuz döngü riskini azaltmak için bir zaman aşımı veya maksimum deneme sayısı eklenmelidir.

```
'DateTime startTime = DateTime.Now;'
```

```
'while (baglantiKurulamadi && (DateTime.Now - startTime).TotalSeconds < 30)'
```

```
'{ ... }'
```

Bu yapı, hem koşul doğru olduğu sürece devam etmeyi hem de maksimum bir süre sonra programı kurtarmayı sağlar.

do-while ve Hata Yönetimi

Kullanıcıdan girdi alırken veya bir servisle bağlantı kurarken hatalar (istisnalar) oluşabilir.

'do { ... } while (hataDevamEdiyor)' yapısı, bir hata oluştuğunda programın çökmesini engellemek ve tekrar deneme fırsatı sunmak için try-catch bloklarıyla birlikte kullanılabilir.

Bu, uygulamaların daha dayanıklı (robust) olmasını sağlar.

Döngü Optimizasyonu: Erken Çıkış

Bir dizide arama yaparken ve istenen öge bulunduğunda, döngünün sonuna kadar devam etmenin bir anlamı yoktur.

Bu tür durumlarda 'break' kullanmak, gereksiz işlemci döngülerini önler ve programın performansını artırır.

Bu, özellikle büyük veri setlerinde (Big Data) çalışan algoritmalar için önemlidir.



Liskov İkame Prensibi ve Döngüler

Nesneye Dayalı Programlama (OOP) bağlamında, döngülerin işlediği koleksiyonlar ve veri yapıları önemlidir. while/foreach döngüleri, soyutlanmış koleksiyonlar (IEnumerable) üzerinde çalışır. Bu, döngü yapısının altta yatan veri yapısı (Array, List, Queue vb.) ne olursa olsun tutarlı bir şekilde çalışabilmesi anlamına gelir (LSP'ye uygunluk).

Görsel Programlamada Kullanıcı Etkileşimi Mantığı

Geleneksel konsol uygulamalarında, program akışı döngüler tarafından yönetilir.

Görsel (WinForms/WPF) programlamada ise akış, kullanıcı eylemleriyle tetiklenen olaylar (events) etrafında döner.

Bu nedenle, bir 'Button click' olayında uzun süreli 'while' döngüleri kullanmak yerine, durum (state) yönetimi ve 'hak sayacı' gibi yaklaşımlar tercih edilir.



ComboBox Stil Çeşitleri

ComboBox kontrolü üç ana stil sunar (DropDownStyle):

1. ****Simple:**** Liste her zaman açıktır, kullanıcı liste dışına da metin yazabilir.
 2. ****DropDown:**** Varsayılan stildir. Kullanıcı listeden seçim yapabilir veya kutuya doğrudan metin yazabilir.
 3. ****DropDownList:**** Kullanıcı sadece listedeki öğelerden birini seçebilir; doğrudan metin girişi yasaktır.
- Bu stil seçimi, uygulamanın gerektirdiği veri bütünlüğüne göre belirlenmelidir.



ComboBox'ın Dizin Bazlı (Index-Based) Çalışması

C# ve birçok modern dil gibi, ComboBox'ın öğeleri de sıfırdan başlayan dizinlerle numaralandırılır.

İlk öğe 0, ikinci öğe 1 dizinine sahiptir.

'SelectedIndex' ve 'Items[index]' kullanımı bu sıfır tabanlı mantığa dayanır. Bu, ComboBox öğelerini for döngüsü ile manipüle etmeyi veya belirli bir öğeyi doğrudan seçmeyi kolaylaştırır.



Dinamik Liste Yönetimi: Items.Remove()

ComboBox'tan öğe kaldırmak için 'Items.Remove()' veya 'Items.RemoveAt(index)' metotları kullanılır.

'Remove()': Belirtilen öğenin kendisini bulur ve kaldırır.

'RemoveAt(index)': Belirtilen dizindeki öğeyi kaldırır.

Bu işlemler, kullanıcı yetkisi değiştikçe veya veri güncellendikçe listedeki seçeneklerin anlık olarak yönetilmesi gerektiğinde önemlidir.



ComboBox ile İlişkili Veri (Associated Data) Tutma

Üniversite seviyesinde, genellikle ComboBox'a string yerine karmaşık nesneler eklenir (örneğin ID'si olan bir 'Müşteri' nesnesi).

Bu durumda, kullanıcıya gösterilen metin ('DisplayMember') ile arka planda kullanılan değer ('ValueMember') birbirinden ayrılır.

Bu ayırım, basit 'Items.Add()' kullanımından daha gelişmiştir ve veritabanı uygulamalarında zorunludur.



Thread Safe (İş Parçacığı Güvenli) Döngü Uygulamaları

Eğer bir 'while' döngüsü arka plan iş parçacığında (thread) çalışıyorsa ve bir WinForms kontrolünü (örneğin ComboBox) güncellemeye çalışıyorsa, bir 'cross-thread operation' hatası oluşur.

Bu, döngülerin UI'yı kilitlenmeden etkilemesi için 'Invoke' veya 'BeginInvoke' gibi Thread Safe yöntemlerin kullanılmasını gerektirir.

Bu, modern C# uygulamasında eş zamanlılık yönetimi açısından kritik bir konudur.



while Döngüsünün Matematiksel Uygulamaları

while döngüleri, özyinelemeli (recursive) fonksiyonların iteratif karşılığı olarak kullanılabilir.

Örneğin, Fibonacci serisi hesaplama veya faktöriyel hesaplama gibi matematiksel problemler, 'for' veya 'while' döngüleri ile özyinelemeden kaçınılarak çözülebilir.

İteratif çözümler genellikle yığın taşması (Stack Overflow) riskini azaltır.



do-while ve Oyun Programlaması

Basit konsol tabanlı oyunlarda, 'do-while' döngüsü oyunun ana menüsünü yönetmek için mükemmeldir.

'do { Oyna } while (kullaniciTekrarOynamakIstiyor);'

Oyunun en az bir kez çalışmasını garantiler, ardından kullanıcıdan tekrar oynayıp oynamayacağını sorar, bu da son test yapısıyla uyumludur.

Koşul İfadelerinin Karmaşıklığı

while ve do-while döngülerindeki koşullar, basit bir boolean değişkeninden çok daha karmaşık olabilir.

'&&' (VE), '||' (VEYA), '!' (DEĞİL) gibi mantıksal operatörler kullanılarak birden fazla durum aynı anda kontrol edilebilir.

Örnek: 'while (hak > 0 && sifreDogruDegil)'.

Bu karmaşık ifadeler, program akışını hassas bir şekilde yönetmeyi sağlar.

Döngü Sınıflaması: Sayaç Kontrollü vs. Sentinel Kontrollü

1. ****Sayaç Kontrollü:**** 'for' döngüsü gibi, önceden belirlenmiş bir sayıya ulaşana kadar çalışan döngülerdir.
2. ****Sentinel Kontrollü:**** 'while' ve 'do-while' döngüleri gibi, belirli bir 'sentinel' (işaretleyici) değer (örneğin 'End-of-File' veya kullanıcı girişi 'exit') veya bir koşul yanlış olana kadar çalışan döngülerdir.
while ve do-while, genellikle sentinel kontrollü programlama için idealdir.

ComboBox'ta Arama ve Filtreleme

Büyük listeler içeren ComboBox'larda kullanıcı deneyimini artırmak için filtreleme önemlidir.

Kullanıcının yazdığı metni kullanarak listedeki öğeleri dinamik olarak filtreleyebiliriz.

Bu işlem, 'Text' özelliği her değiştiğinde (TextChanged olayı) listeyi güncelleyen bir 'while' veya 'for' döngüsü gerektirir.



ComboBox'ın Okunabilirlik Özellikleri

ComboBox'ın 'Sorted' özelliği 'True' olarak ayarlanırsa, öğeler eklenme sırasına bakılmaksızın alfabetik olarak sıralanır.

Bu özellik, kullanıcıların uzun listelerde aradıkları öğeyi daha kolay bulmalarını sağlar.

Dinamik olarak öğe eklerken (döngü ile), bu özelliğin 'True' olması önemlidir.



C# Standart Kütüphanesi ve Döngü İfadeleri

Modern C#'ta, basit döngü operasyonlarının çoğu (filtreleme, sıralama, arama) LINQ ifadeleri ile daha deklaratif (bildirimsel) bir şekilde gerçekleştirilebilir.

LINQ, arka planda hala iterasyon kullanır, ancak bu iterasyonun yönetimini geliştirici üzerinden alır.

Üniversite öğrencileri, döngü temellerini öğrendikten sonra LINQ'e geçmelidir.



do-while ve İzin Kontrolü

Finansal veya kritik sistemlerde, bir işlemi gerçekleştirmeden önce kullanıcıdan PIN veya ek onay almak gerekebilir.

'do-while' yapısı, kullanıcının doğru onayı vermesi koşuluna bağlı olarak işlemi tekrarlatmak için güçlü bir araçtır.

Bu, güvenliği artırır ve yanlışlıkla yapılan işlemleri önler.



while Döngüsü ve Network (Ağ) Programlama

Sunucu uygulamalarında, istemcilerden gelen verileri sürekli dinlemek gerekir. Bir 'while(true)' döngüsü (genellikle bir arka plan iş parçacığında), bir ağ soketinin sürekli olarak yeni paketler için beklemesini sağlar. Bu kritik döngü, sunucu uygulamasının ana çalışma mantığını oluşturur ve ağ kesintisi gibi durumlarda güvenli 'break' mekanizmalarını gerektirir.



ComboBox: Veri Tipi Yönetimi

'Items.Add()' metodu bir 'object' parametresi alır.
String veya int gibi değer tipleri eklendiğinde, bunlar otomatik olarak 'object' tipine dönüştürülür (boxing).
'SelectedItem' veya 'Items[index]' geri alındığında, tekrar orijinal tipine dönüştürülmelidir (unboxing veya casting). Ör:
'(string)comboBox1.SelectedItem';
Bu, ComboBox'ın esnek yapısının bir sonucudur.

Döngü Yapılarında 'goto' Kullanımı

'goto' ifadesi, program akışını doğrudan belirli bir etikete atlatır. Teorik olarak, iç içe döngülerden hızlıca çıkmak için kullanılabilir (birden fazla 'break' yerine). Ancak, 'goto' kullanımı kodun okunabilirliğini ve bakımını ciddi şekilde zorlaştırdığı için, modern akademik C# programlamada ****şiddetle tavsiye edilmez****.

while Döngüsü ve Boolean Bayrakları (Flags)

while döngüsünün koşulu genellikle basit bir sayaçtan ziyade, bir boolean bayrağına bağlıdır.

```
'bool devam = true;'
```

```
'while (devam)'
```

```
{'
```

```
' // bir koşulda devam = false yapılır'
```

```
'}'
```

Bu, döngü sonlandırma mantığının kodun farklı yerlerinden (örneğin bir metod çağrısından) tetiklenmesini sağlar.

do-while: Menü Kullanımı Örneği Detaylı

Konsol uygulamalarında, ana menüyü tekrar tekrar göstermek ve kullanıcıdan geçerli bir seçim alana kadar döngüde kalmak için do-while idealdir.

'do' bloğu menüyü gösterir ve girdiyi alır.

'while' koşulu, kullanıcının seçimi geçerli olmadığı veya çıkış seçeneği seçilmediği sürece 'true' döner.

Bu, en az bir kez menünün görüntülenmesini garanti eder.



ComboBox ve Geniřletilebilirlik (Inheritance)

Üniversite projelerinde, standart ComboBox'ın yetenekleri yetersiz kalabilir. Bu durumda, geliřtiriciler kendi özel ComboBox kontrollerini oluřturmak için 'ComboBox' sınıfından türetme (inheritance) yapabilirler. Bu özelleřtirilmiř kontroller, örneğın otomatik filtreleme veya veritabanından dinamik yükleme gibi ek mantıkları içerir.

while Döngüsünde Değişken Kapsamı (Scope)

while döngüsünün koşulunda kullanılan veya döngü içinde güncellenen sayaçlar veya boolean bayrakları, döngü bloğunun dışında tanımlanmalıdır. Eğer değişken döngü içinde tanımlanırsa, her yeni iterasyonda sıfırlanır ve döngü dışından erişilemez.

Örnek: `int i = 0; while (i < 10) { ... }`

do-while ve API Eriřimleri

Bazı web servisleri veya API'ler, ilk çağrıyı yapmadan durum (state) bilgisi döndürmez.

'do-while' bu tür durumlar için idealdir: Önce servisi çağır ('do'), ardından gelen yanıtta hata olup olmadığını veya devam etme koşulunun sağlanıp sağlanmadığını kontrol et ('while').

Bu, ilk denemenin mutlaka gerçekleşmesini gerektiren protokollerle uyumludur.



ComboBox'ta Metin Özelliği ve Tür Güvenliği

Eğer ComboBox stili metin girişine izin veriyorsa ('DropDown'), kullanıcının girdiği metin her zaman string'dir. Ancak bu metin sayısal bir değer olarak kullanılacaksa, mutlaka 'int.TryParse()' gibi metotlarla tür dönüşümü ve doğrulama yapılmalıdır. Bu, uygulamanın çökmesini (crash) önleyen temel bir güvenlik adımıdır.

while Döngüsü ve Dosya İşlemleri

Dosya okuma işlemlerinde 'StreamReader.ReadLine()' metodunun dönüş değerini kontrol etmek için 'while' yaygın kullanılır.

```
'while ((line = reader.ReadLine()) != null)'
```

Bu döngü, dosyanın sonuna ('null') ulaşılanaya kadar çalışmaya devam eder ve her adımda bir satırı işler.

Bu, bilinmeyen boyuttaki veri akışlarını yönetmek için temel bir desen (pattern) oluşturur.



ComboBox'ın Accessibility (Erişilebilirlik) Özellikleri

Üniversite projelerinde, UI kontrollerinin erişilebilirliği dikkate alınmalıdır. ComboBox'ın 'AccessibleName' ve klavye kısayolları, görme engelli kullanıcılar veya fare kullanamayanlar için önemlidir. Döngüler aracılığıyla dinamik olarak doldurulan listelerin, erişilebilirlik standartlarına uygun olduğundan emin olunmalıdır.



while ve do-while Döngülerinin Bellek Yönetimi

Döngü bloğu içinde tanımlanan her yerel değişken, o iterasyon sona erdiğinde bellekten silinmeye (Garbage Collection) aday olur.

Uzun süren döngülerde, gereksiz büyük nesnelerin döngü içinde sürekli yeniden oluşturulması performansı düşürebilir ve bellek basıncını artırabilir. Bu nedenle, döngü içinde sık kullanılacak nesneler döngü dışında tanımlanmalıdır (ön yükleme).



C# 8.0 ve Sonrası: foreach ve IEnumerable

Modern C# sürümlerinde, ağ veya veritabanı gibi I/O işlemlerini asenkron olarak döngüye almak için 'await foreach' yapısı tanıtılmıştır. Bu, UI'ı kilitlemeden ('while' döngüsünün neden olduğu problem) büyük veri akışlarını işlemeyi mümkün kılar. Bu, klasik 'while' döngüsünün asenkron programlamaya adaptasyonudur.

Proje Örneği Detaylandırma: Şifre Doğrulama

Görsel programlamada, şifre denemesi genellikle şu adımlarla yönetilir:

1. Giriş düğmesi tıklanır.
2. Statik değişken 'denemeSayisi' kontrol edilir.
3. 'if (şifre_yanlış)': Uyarı gösterilir, 'denemeSayisi' azaltılır.
4. 'if (denemeSayisi <= 0)': Buton devre dışı bırakılır (do-while'in bitiş koşulu simülasyonu).

Bu yaklaşım, UI kilitlenmesini engellediği için 'do-while' yerine tercih edilir.

ComboBox: SelectedItem'in Casting Zorunluluğu

'SelectedItem' özelliği bir 'System.Object' döndürür.

Eğer eklediğimiz öğeler string ise, bunları kullanmadan önce string türüne açıkça dönüştürmek gerekir.

Örnek: `'string secilenSehir = comboBox1.SelectedItem as string;'`

'as' operatörünü kullanmak, dönüşüm başarısız olursa null döndürdüğü için daha güvenli bir yöntemdir.

Döngü İçinde Metot Çağrısı

Karmaşık işlemler döngü içinde tekrarlanacaksa, bu işlemleri ayrı bir metoda taşımak iyi bir programlama pratiğidir (Fonksiyonel Dekompozisyon).
while döngüsü içinde bu metodu çağırmak, hem döngü bloğunu temiz tutar hem de test edilebilirliği artırır.



Döngü Optimizasyonu: Dizilerin Ön Bellek (Cache) Erişimi

Yüksek performanslı kodlamada, döngülerin bellek erişim düzeni önemlidir. Diziler üzerinde ardışık (sequential) erişim (for veya while ile) genellikle CPU ön belleği (cache) tarafından desteklenir ve rastgele erişimden daha hızlıdır. Bu, büyük veri yapılarında döngülerin nasıl yazılacağını etkileyen derin bir bilgisayar mimarisi konusudur.



ComboBox ve Kullanıcı Girişi Güvenliği (Sanitization)

Eğer ComboBox, kullanıcı girişine izin veren stildeyse ve bu metin doğrudan bir veritabanı sorgusunda kullanılıyorsa, SQL Injection riski oluşur. Üniversite düzeyinde bilinmesi gereken en önemli güvenlik konularından biri: Döngülerden veya UI kontrollerinden gelen tüm girdilerin parametrelendirilmesi veya temizlenmesi (sanitization) zorunludur.



while Döngüsü ve Üretici/Tüketici (Producer-Consumer) Deseni

while döngüsü, bir kuyruktan (queue) veri tüketen thread'ler (iş parçacıkları) için temel yapıdır.

'while (queue.Count > 0 || isProducing)' döngüsü, kuyrukta işlenecek veri olduğu sürece çalışır.

Bu, modern, çok iş parçacıklı sistemlerde yaygın bir mimari desendir.

ComboBox'ta Varsayılan Değer Belirleme

Liste doldurulduktan sonra, kullanıcının boş bir kutu görmemesi için genellikle bir varsayılan öge seçilir.

Bu, ya 'SelectedIndex = 0' (listenin ilk ögesi) ya da 'Text = 'Lütfen Seçiniz"' gibi bir kılavuz metin atayarak yapılır.

Bu, formun başlangıç durumunun (initial state) doğru ayarlanmasını sağlar.



Döngü Yapılarının Hata Ayıklaması (Debugging)

Döngü hatalarını (örneğin sonsuz döngü veya yanlış koşul) bulmak için hata ayıklama araçları hayati öneme sahiptir.

****Kesme Noktaları (Breakpoints):**** Döngü başlangıcına veya kritik koşul kontrol satırına konulur.

****Gözlem Pencereleri (Watch Windows):**** Koşul değişkenlerinin ('i', 'hak', 'koşul') her iterasyonda nasıl değiştiğini takip etmek için kullanılır.

do-while: Güncel Uygulamalar ve Web Programlama

Web tabanlı uygulamaların JavaScript (AJAX) tarafında da do-while benzeri yapılar kullanılır.

Örneğin, sunucudan bir yanıt alana kadar belirli bir işlemi tekrar etme (polling) mantığı, 'do-while' yapısının işlevsel karşılığıdır.

Bu, backend'den bağımsız olarak frontend'deki kullanıcı etkileşimi döngülerini yönetir.

ComboBox'ın Clear Metodu

Bir ComboBox'ın tüm öğelerini kaldırmak için 'Items.Clear()' metodu kullanılır. Bu işlem, kullanıcı bir seçim yaptığında veya uygulamanın durumu değiştiğinde (örneğin ülke seçimine göre şehir listesini güncellemek) dinamik olarak listenin sıfırlanması gerektiğinde önemlidir. Bu, yeni veri setini döngü ile tekrar doldurmadan önce gereklidir.



Döngü Tasarımında İsimlendirme Kuralları

Üniversite projelerinde, döngü değişkenlerinin (özellikle while ve do-while'da koşulu kontrol edenler) anlamlı isimlendirilmesi önemlidir. Basit sayaçlar için 'i', 'j' kabul edilebilirken; 'hak', 'baglantiDurumu', 'dosyaSonu' gibi anlamlı boolean veya sayısal değişkenler, döngünün amacını hızla anlamayı sağlar.



while ve do-while Döngülerinin Eşdeğerliği

while döngüsü ile her zaman do-while döngüsünün mantığını simüle etmek mümkündür, ancak bu daha karmaşık bir kod gerektirir.

Örn: 'bool ilkGecis = true; while (koşul || ilkGecis) { ... ilkGecis = false; }'

Tersine, bir do-while döngüsünün hiç çalışmama garantisini sağlamak zordur. Bu nedenle, her döngü yapısı, amacına en uygun olan senaryoda kullanılmalıdır.



ComboBox'ta SelectedValue Özelliği

Eğer 'DisplayMember' ve 'ValueMember' özellikleri ayarlanmışsa (yani karmaşık nesneler eklenmişse), 'SelectedValue' özelliği, seçilen öğenin 'ValueMember' tarafından tanımlanan değerini döndürür.

Bu, arka uçta (back-end) ID veya anahtar (key) değerini hızlıca almak için çok kullanışlıdır.



Koleksiyonlardan Öğe Silme Sorunları

Bir 'while' döngüsü veya 'for' döngüsü ile bir koleksiyon üzerinde dönerken aynı anda bu koleksiyondan öğe silmek (örneğin 'List.Remove()') 'InvalidOperationException' hatasına neden olabilir.

****Çözüm:**** Ya geriden başlayarak silme işlemi yapılır ya da silinecek öğelerin dizinleri toplanıp döngü bittikten sonra silme işlemi gerçekleştirilir.

ComboBox'ta Otomatik Tamamlama (AutoComplete)

ComboBox'ın 'AutoCompleteMode' ve 'AutoCompleteSource' özellikleri ayarlanarak otomatik tamamlama özelliği etkinleştirilebilir. Bu, kullanıcının kutuya yazmaya başladığında listedeki olası eşleşmelerin önerilmesini sağlar. Bu özellik, uzun listeler içeren ComboBox'lar için temel bir kullanıcı kolaylığıdır.



Program Kontrol Akışı (Control Flow) Özeti

Programlamada kontrol akışı, hangi kod satırlarının hangi sırayla çalışacağını belirler.

'if-else' koşullu dallanmayı sağlar.

'for', 'while', 'do-while' ise kod bloklarının tekrarlanmasını sağlar.

Bu yapıların doğru entegrasyonu, herhangi bir karmaşık algoritmanın başarılı bir şekilde uygulanmasının temelini oluşturur.



while ve do-while: Gömülü Sistemler (Embedded Systems)

Gömülü sistemlerde (mikrodenetleyicilerde) ana çalışma mantığı genellikle sonsuz bir 'while(true)' döngüsü üzerine kuruludur.

Bu döngü, cihazın sensörleri sürekli okumasını veya durum değişikliklerini beklemesini sağlar.

Bu kritik döngü, cihazın arızalanmaması için çok dikkatli optimize edilmeli ve kilitlenmeye karşı koruma mekanizmaları (watchdog timer) içermelidir.



Örnek Proje Analizi: Bağlantı Yeniden Deneme Mantığı

Basit bir ağ bağlantısı denemesi:

```
'int denemeSayisi = 0;'  
'while (baglantiBasarisiz && denemeSayisi < 5)'  
'{'  
'  DenemeYap();'  
'  denemeSayisi++;'  
'}'
```

Bu yapı, 'while' döngüsünün güçlü ve kontrol edilebilir bir deneme mekanizması oluşturma yeteneğini gösterir.

ComboBox Kontrolünün Alternatifleri

ComboBox, tek bir seçim için yer tasarrufu sağlarken, diğer listeleme kontrolleri farklı ihtiyaçları karşılar:

****ListBox:**** Kullanıcıya birden fazla öğeyi aynı anda gösterir, birden çok seçim yapabilir.

****CheckedListBox:**** Öğelerin yanına onay kutuları koyarak çoklu seçim için özel olarak tasarlanmıştır.

Doğru kontrol seçimi, kullanıcı arayüzü tasarımının önemli bir parçasıdır.



Döngü Performansı: Ön Yükleme (Pre-loading) Stratejisi

Eğer döngü içinde sürekli olarak bir metot çağrılıyor ve bu metot her zaman aynı sonucu döndürüyorsa (saf fonksiyon), metot çağrısı döngüden önce yapılmalıdır.

Örn: 'int limit = Dizi.Length; while (i < limit)' yerine 'while (i < Dizi.Length)' yazmak. Küçük optimizasyonlar gibi görünse de, milyonlarca iterasyonda büyük performans farkı yaratabilir.



ComboBox ve DataGridView Entegrasyonu

Gelişmiş WinForms uygulamalarında, DataGridView (Veri Izgarası) içindeki hücreler bir ComboBox olarak ayarlanabilir.

Bu, kullanıcının tabloda düzenleme yaparken önceden tanımlanmış seçenekleri seçmesini sağlar.

Bu ComboBox hücrelerinin listeleri, yine döngüler veya Data Binding ile dinamik olarak doldurulur.

Özet ve Değerlendirme: while ve do-while

while: Önce koşul kontrol edilir. Koşul başlangıçta yanlışsa hiç çalışmaz (0 veya daha fazla iterasyon).

do-while: Önce gövde çalışır, sonra koşul kontrol edilir. En az bir kez çalışması garantidir (1 veya daha fazla iterasyon).

Hangi döngünün kullanılacağına karar verirken, en az bir kez çalışmanın zorunlu olup olmadığına bakılmalıdır.

Özet ve Değerlendirme: ComboBox Kontrolü

ComboBox, kullanıcı arayüzünde sınırlı seçenekleri düzenli bir şekilde sunar. Temel Özellikler: 'Text', 'SelectedIndex', 'SelectedItem'. Öğe Ekleme: Statik olarak 'Items.Add()' ile veya dinamik olarak 'for'/'foreach' döngüleriyle dizilerden/listelerden yapılır. Etkin programlama için döngülerin UI kontrolüyle entegrasyonu esastır.

Gelecek Konulara Hazırlık

Gelecek haftalarda, öğrendiğimiz döngü yapılarını daha karmaşık veri yapıları (Sözlükler, Kuyruklar, Yığınlar) üzerinde uygulamayı göreceğiz.

****foreach**** döngüsünün koleksiyonlar için sağladığı avantajları ve Data Binding kavramını derinlemesine inceleyeceğiz.

Bu hafta edindiğiniz döngü ve kontrol bilgileri, tüm ileri konuların temelini oluşturmaktadır.



Koşullu Tekrarlar ve Kontrol Akışı

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

